

COMPLETE PROJECT STUDY GUIDE

AI-Powered Mobile Test Orchestration Platform

Everything that exists in this project today — architecture, modules, flows, AI layer, data model, current status, known gaps, and discussion prep.

Codename: Vyapy QA / AutomationTesting Platform

Version: 1.0.0-rc (Release Candidate)

Branch: feature/android-bridge-and-platform-fixes

Date of snapshot: 3 August 2026

Runtime: macOS · Python 3.10+ · Node 18+ · Xcode / iOS Simulators

Size: 134 Python files · ~24,700 Python LOC · ~9,800 TS/TSX LOC

Table of Contents

- | | |
|---|--|
| 1. The 60-Second Pitch | 15. Core Flow F — The External Vya Bot |
| 2. The Problem & Why This Exists | 16. The AI Layer |
| 3. The Domain: The Vya Restaurant Product | 17. The Intelligence Layer (Differentiators) |
| 4. System Architecture | 18. Device & Simulator Hardening |
| 5. Technology Stack | 19. Reporting, Notifications & Integrations |
| 6. Repository Map — Module by Module | 20. Operations — Running the Platform |
| 7. The Data Model (24 Tables) | 21. Current Status — Honest Scorecard |
| 8. The API Surface (110+ Endpoints) | 22. Real Numbers From the Live Database |
| 9. The Dashboard (14 Pages) | 23. Known Issues & Root Causes |
| 10. Core Flow A — Project Preparation | 24. The Roadmap — Layered Automation |
| 11. Core Flow B — Plain-Language Scenario Run | 25. Design Decisions & Trade-offs |
| 12. Core Flow C — The Recorder | 26. Group Discussion Q&A Prep |
| 13. Core Flow D — Cross-App Orchestration | 27. Glossary |
| 14. Core Flow E — Autonomous PR Testing | |

1. The 60-Second Pitch

This is an **enterprise-grade, AI-assisted mobile QA platform**. It clones mobile app repositories, builds them, installs them on iOS simulators (and Android devices), drives them through real user journeys written in **plain English**, and reports back with AI-generated root-cause analysis. It plugs into GitHub so that opening a pull request automatically triggers a smart-selected regression run and posts a verdict back on the PR.

The three things that make it different from "just Appium + a CI job":

1. **Plain-language authoring.** A non-technical QA person writes `open the app`, `book a table`, `verify the bill is correct`. The engine resolves each step against the *live accessibility tree* of the running app — no hand-written selectors.
2. **Cross-app testing.** The product under test is not one app; it's a Consumer app and a Business app (with Waiter and Kitchen roles). The platform drives **two simulators at once** and synchronises them at handoff points, so a single test verifies an end-to-end order lifecycle across apps, accounts and devices.
3. **An intelligence layer.** Self-healing locators that persist across runs, graph-driven smart test selection from a git diff, flaky detection, visual regression, performance scoring, and LLM root-cause analysis — all local-first (Ollama, no API key required).

The one-line status

The platform and its features are **built and working**; live iOS execution is **functional but slow on the 2019 Intel Mac** (WebDriverAgent build time); what remains is the sequential cross-app flow, the Business-app scenario library, and production hardening (Postgres, containerisation, hosting). As a product: roughly **80% there**.

2. The Problem & Why This Exists

2.1 The pain being solved

PAIN	HOW THE PLATFORM ANSWERS IT
Mobile regression testing is manual, slow and repeated every release.	Record once → replay forever, on real simulators, with evidence.
Appium scripts break constantly when developers rename an element.	Self-healing locators: remembered locator tried first; on drift, re-derive and re-learn.
QA people can't write code, developers don't want to write tests.	Steps are plain English; the platform generates the real pytest/Appium script from what actually worked.
Running the whole suite on every PR is too slow.	Graph-driven smart selection: from the git diff, work out the blast radius and run only the scenarios that cover it.
A failed run gives you a stack trace, not an explanation.	RCA reports: root cause, category, affected modules, confidence, suggested fix, severity.
The product is multi-app — a bug only appears when Consumer and Business interact.	Cross-app orchestration with both-must-pass gating on every scenario.
Dashboards lie — a suite with zero tests shows green.	"Honest verdicts": a run with no scenarios reports "no tests", never a false green.

2.2 Who uses it

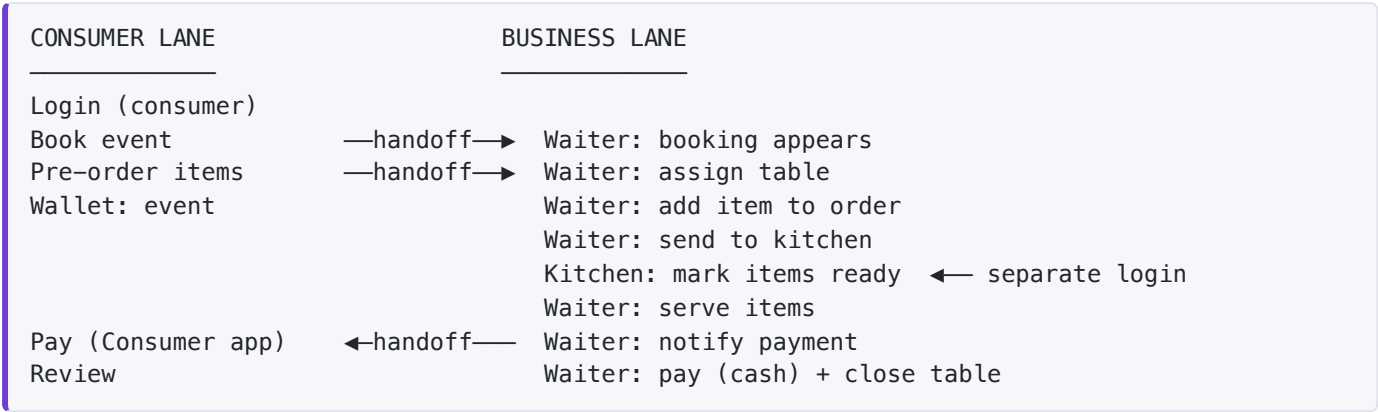
- **QA engineer** — records flows, builds scenarios, runs suites, reads reports.
- **Developer** — opens a PR, gets an automatic QA verdict as a PR comment.
- **Manager / PM** — reads AI narrative summaries, trends, pass-rate charts, coverage maps.
- **Platform admin** — device registry, agents, ops health, alerts, audit log.

3. The Domain: The Vya Restaurant Product

You cannot explain the platform without explaining what it tests. **Vya** is a restaurant booking & dining product made of two React Native apps:

APP	ROLE	TYPICAL DEVICE	WHAT IT DOES
Consumer app (vya-consumer)	Diner	iPhone simulator	Login, book an event, invite guests/participants, pre-order items, wallet, pay, review.
Business app (vya-business)	Waiter	iPad simulator	See bookings, assign table, take/add order items, send to kitchen, serve, take payment, close table.
Business app (same binary)	Kitchen	iPad (2nd account/device)	Receive orders, mark items ready.
Vya web app	Web	—	Registered as a project; web surface of the same product.

3.1 The "spine" — the core cross-app happy path



Everything else in the ~150-scenario coverage matrix is a **variation** of this spine: booking variations (guests, participants, indoor/outdoor, duration), ordering variations (modifiers, variants, special instructions, splits, tips), preorder variations, payment variations (cash / e-payment / voucher / split / host-pays-all), cancellations, and the long tail (contacts, subscriptions, blocked users, filters, PDFs, complimentary items, voids).

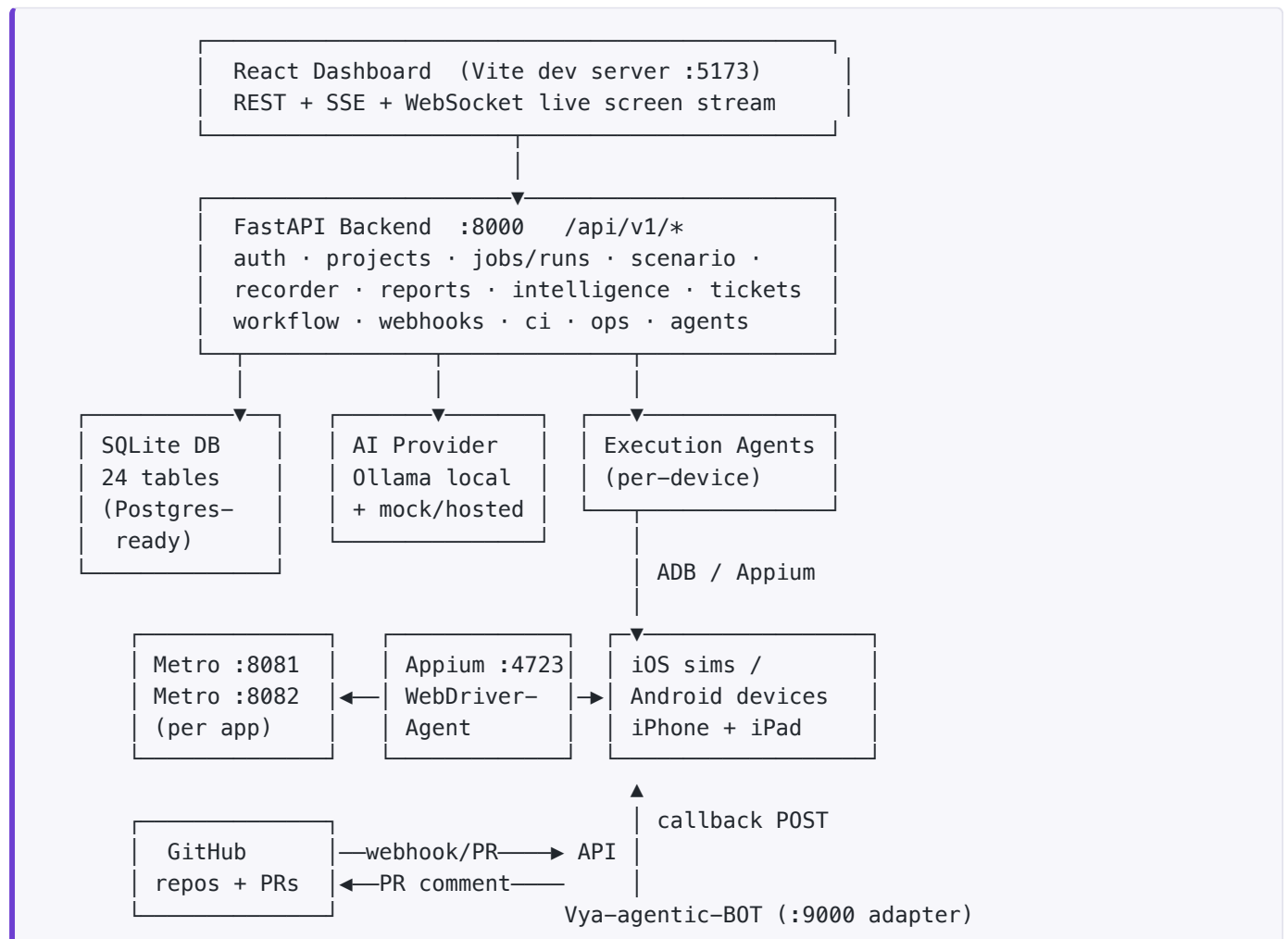
Deliberately marked MANUAL — never false-green
Split-via-swipe/toggle/modal gestures, QR-code booking, location search/add/current, reviews (accessibility-ID gaps), search-from-home in the Consumer app, and payment when a due/outstanding amount exists. These are tagged as un-automatable so the platform never claims coverage it doesn't have.

4. System Architecture

Four cooperating parts:

PART	TECH	RESPONSIBILITY
Dashboard	React 19 + Vite + TypeScript	The UI: projects, runs, scenarios, recorder, PRs, graphs, workflow map, AI chat, reports, settings.
API / Orchestrator	FastAPI (Python)	The brain: auth, project prep, builds, run orchestration, AI, device registry, webhooks, tickets.
Execution layer	Appium + <code>xcrun simctl</code> / <code>adb</code> + Metro	Actually drives simulators/devices and the app under test.
External bot	Vya-agentic-BOT (separate repo, on a Mac)	Runs the ~70-scenario restaurant suite via <code>idb</code> and reports results back to the API.

4.1 Runtime topology



4.2 Port map (memorise this — it comes up in every demo)

PORT	SERVICE	NOTE
8000	FastAPI API	<code>/api/v1/*</code> , <code>/docs</code> for Swagger
5173	Vite dashboard	The only URL a user opens
4723	Appium server	WebDriver protocol; auto-started by <code>start.sh</code>
8081 / 8082	Metro bundler	Consumer / Business — each RN app needs its own port or they collide
8083 / 8084	Metro (staging)	Staging consumer / business builds
9000	Bot adapter	Queues jobs for the external Vya bot
11434	Ollama	Local LLM; optional, disabled during demos for speed

5. Technology Stack

Backend

- Python 3.10+, **FastAPI**, Uvicorn
- **SQLAlchemy** ORM + Alembic migrations
- SQLite default (`test_automation_new.db`); `DATABASE_URL` switches to Postgres (psycopg2 installed)
- JWT auth (`python-jose`) + `passlib[bcrypt]`
- Appium-Python-Client (WebDriver)
- `httpx` for LLM + GitHub calls
- Redis referenced, not required

Execution

- **iOS:** `xcodebuild`, `xcrun simctl`, CocoaPods, WebDriverAgent/XCUITest
- **Android:** Gradle `assembleDebug`, `adb install -r`, UiAutomator2
- **React Native:** Metro bundler with per-app port routing + `RCT_jsLocation`
- **Bot:** Facebook `idb` for coordinate-level control

Frontend

- **React 19** + **Vite 8** + TypeScript 6
- React Router 7 (14 routes)
- **Recharts** — trends, pass-rate, performance charts
- **D3** — dependency & workflow graphs
- **Monaco** — in-browser code editor
- `react-markdown`, `lucide-react`, `date-fns`
- `oxlint` for linting

AI / Intelligence

- **Ollama** local (`llama3.2`, optionally `llama3.1:8b`) — no API key
- Provider abstraction with mock fallback + hosted LLM support
- **Graphify** (tree-sitter AST) for multi-language dependency graphs
- **OpenCV** pixel-diff for visual regression

Why local LLM? (ADR-001)

A deliberate architecture decision: the platform must run with **zero API keys and zero data leaving the machine** — test logs contain app internals and credentials. Hence Ollama-first behind a provider abstraction, with a mock provider so every AI feature degrades gracefully instead of erroring when no model is running.

6. Repository Map — Module by Module

```
AutomationTesting/
├── start.sh · stop.sh · status.sh · start-backend.sh  ← one-command lifecycle
├── install.py                                         ← environment wizard
├── requirements.txt · pyproject.toml · docker-compose.yml
├── alembic/                                           ← DB migrations
├── automation/                                       ← THE BACKEND (134 py files)
├── automation/dashboard/                             ← THE FRONTEND (React)
├── demo-mobile-tests/                               ← sample test repo (POM style)
├── docs/                                              ← 9 markdown docs + PDFs
├── graphify-out/                                     ← cached AST knowledge graph
├── repos/                                             ← cloned app repos (per project UUID)
├── reports/ · logs/ · baselines/                     ← run artefacts
└── test_automation_new.db                           ← live SQLite database
```

6.1 The `automation/` packages

PACKAGE	FILES	WHAT IT DOES
<code>api/</code>	24	FastAPI app (<code>main.py</code>) + all v1 routers. Wires CORS, JWT, global structured error handler, WebSocket screen stream, startup hooks (DB init, orphan-run reaping, ops monitor, PR poller, Metro watchdog).
<code>intelligence/</code>	19	The brain of test execution & analysis: <code>scenario_runner</code> (1,389 LOC — NL→Appium), <code>learned_locators</code> , <code>flaky_detector</code> , <code>hybrid_impact_analyzer</code> (516 LOC), <code>visual_regression</code> , <code>impact_predictor</code> , <code>impact_selection</code> , <code>pr_planner</code> , <code>pr_autotest</code> , <code>git_analyzer</code> , <code>element_catalog</code> , <code>locator_catalog</code> , <code>bill_validator</code> , <code>cross_app_analyzer</code> , <code>memory</code> , <code>chat</code> , <code>analytics</code> , <code>prediction</code> , <code>recommendation</code> .
<code>ai/</code>	13	LLM provider abstraction (<code>provider.py</code> , <code>providers.py</code>), <code>prompts.py</code> , and services: <code>chat</code> , <code>execution</code> , <code>generator</code> , <code>intelligence</code> , <code>knowledge</code> , <code>reporting</code> , <code>summary</code> .
<code>dashboard/</code>	26	React SPA — 14 pages, 8 shared components, typed API client with SSE readers.
<code>appium_service/</code>	10	Appium server lifecycle: <code>manager</code> , <code>session_manager</code> , <code>port_manager</code> , <code>health_monitor</code> , <code>log_manager</code> , <code>driver_factory</code> , <code>config</code> , providers, its own router.
<code>database/</code>	6	SQLAlchemy <code>models.py</code> (24 tables, 474 LOC), <code>config.py</code> (engine + <code>create_all</code> + lightweight column-add migration), <code>database.py</code> (query helpers).
<code>projects/</code>	5	<code>preparation.py</code> (the prep pipeline), <code>builder.py</code> (1,473 LOC — build/install/launch/device resolution/Metro watchdog), <code>repository.py</code> (git), <code>detector.py</code> (project type), <code>config.py</code> .
<code>scenarios/</code>	5	<code>cross_app_orchestrator.py</code> (460 LOC — two sims at once), <code>cross_app_config.py</code> (device/role/credential mapping), <code>cross_app_flows.py</code> , <code>run_full_scenario.py</code> , plus a YAML suite definition.
<code>android_bot/</code>	5	Bridge to the external Vya-agentic-BOT: <code>adapter.py</code> , <code>queue_manager.py</code> , <code>scenario_reporter.py</code> , <code>run_vya_bot_suite.py</code> , and <code>TANISH_BOT_PATCH.md</code> (the bot-side integration contract).
<code>device_manager/</code>	6	Device discovery (<code>discovery/ios.py</code> , <code>discovery/android.py</code>) + in-memory registry with 90-second staleness expiry.
<code>performance/</code>	3	<code>collector.py</code> (CPU/memory/FPS/launch-time sampling on a background thread) + <code>api_interceptor.py</code> (API latency capture).
<code>workflow/</code>	2	<code>catalog.py</code> (321 LOC) — the single source of truth for the ~150-scenario coverage matrix and the app "spine" with cross-app handoff edges.
<code>streaming/</code>	3	<code>screen_capture.py</code> (simctl screenshots at ~4 FPS) + <code>ws_manager.py</code> (frame buffer per run).
<code>integrations/</code>	4	<code>github.py</code> (PR metadata, clone URLs, commit status), <code>jira.py</code> (auto-tickets), <code>pr_comment.py</code> (verdict comment bot).
<code>notifications/</code>	3	<code>notifier.py</code> , <code>dispatch.py</code> , <code>realtime_alerts.py</code> — Slack on failure/verdict.
<code>evidence/</code>	3	<code>collector.py</code> (screenshots + logs per run) and <code>pii_scrub.py</code> (redact before storing).
<code>reporting/ + reports/</code>	4	HTML/Markdown/PDF report engine + scenario report renderer.

ci_cd/	2	<code>pr_poller.py</code> (checks open PRs every 5 min) + a real GitHub Actions workflow template.
knowledge/	3	<code>default.md</code> , <code>vya_scenario_library.md</code> , <code>learned_locators.json</code> — the persisted locator memory and the AI's grounding corpus.
agent/ , runner/ , test_runner/ , plugins/	5	The distributed execution agent (polls jobs, heartbeats, discovers devices) and the framework plugin system (<code>appium_framework.py</code> + <code>base.py</code>).
auth/ , utils/ , ops/	4	JWT security, pre-flight environment validator, secret-masking log filter, ops health monitor.

7. The Data Model (24 Tables)

TABLE	HOLDS
users	Accounts + JWT auth. Roles: admin / qa / developer / viewer.
application_groups	Logical groupings of projects (e.g. "Vya prod", "Vya staging") for impact scope.
test_projects	A repo under test: git URL, branch, platform, app_bundle_id , clone status.
test_runs	One execution: status, device, platform, timings, bot_type , branch, commit, triggered_by , job_state .
scenario_results	Per-scenario cross-app results — Consumer + Business statuses merged into one row with a both-must-pass gate.
saved_scenarios	Reusable user-built scenarios: name, project, device, ordered plain-language steps, coverage tags.
rca_reports	AI root-cause analysis: root cause, category, affected modules, confidence, impact, suggested fix, priority, severity, LLM provider/model.
evidence_bundles	Screenshots + logs per run (PII-scrubbed).
dependency_graphs	Cached module dependency graphs per project.
module_stability	Per-module flakiness / stability metrics.
execution_agents	Registered runner agents + last heartbeat (feeds the device registry).
performance_metrics / performance_summaries	CPU, memory, FPS, API latency samples + the scored per-run summary.
visual_regression_results	Pixel-diff percentage vs baseline, per screen, with severity.
tickets	Pasted issues linked to a PR + the scenarios that verify them; auto-tested on push.
chat_sessions / chat_messages	AI Chat history (memory-backed).
ai_recommendations	AI-suggested actions surfaced on the dashboard.
user_feedback	Thumbs up/down on AI output — the quality feedback loop.
project_settings	Per-project configuration overrides.
audit_logs / system_alerts	Operations trail + alerting.

Migration strategy

Base.metadata.create_all creates missing tables at startup, plus a **lightweight column-add migration** for existing tables — so a schema change never requires a manual migration during development. Alembic exists for the eventual Postgres move.

8. The API Surface (110+ Endpoints)

Everything under `/api/v1`. Auth is JWT via a `get_current_user` dependency on most routers. Three deliberate exceptions: **GitHub webhooks** (GitHub calls them), the **bot scenario-result callback** (guarded by `X-Bot-Secret`), and the **agent frame upload** (guarded by `X-Agent-Secret`, constant-time compared).

ROUTER	PREFIX	NOTABLE ENDPOINTS
auth	<code>/auth</code>	<code>POST /login</code> , <code>GET /me</code> , <code>POST /seed</code>
projects	<code>/projects</code>	CRUD · <code>/{{id}}/clone</code> · <code>/pull</code> · <code>/validate</code> · <code>/prepare</code> (+ status) · <code>/generate-yaml</code> · <code>/performance-trends</code>
pull_requests	<code>/projects</code>	<code>GET /{{id}}/pulls</code> · <code>POST /{{id}}/pulls/{{n}}/test</code>
webhooks	<code>/webhooks</code>	GitHub push / PR events → queue a run (no JWT)
jobs + runs	<code>/jobs</code> , <code>/runs</code>	<code>POST /poll</code> · <code>/{{job}}/status</code> · <code>/stream</code> · <code>/evidence</code> · <code>POST /{{run}}/scenario-result</code> (bot callback) · <code>/trigger-android-bot</code> · <code>GET/POST /cross-app</code> + <code>/cross-app/config</code> · <code>POST /runs/vya-bot</code>
scenario	<code>/scenario</code>	<code>POST /run</code> · <code>/run/stream</code> (SSE, live per-step) · <code>capture-locators</code> · <code>generate-scripts</code>
scenarios	<code>/scenarios</code>	Saved-scenario CRUD · <code>GET /devices</code> · Run All
recorder	<code>/recorder</code>	Start/stop session · frame · hit-test-and-tap · record step · save as scenario
intelligence	<code>/intelligence</code>	<code>GET /runs/{{id}}/rca</code> · <code>flaky/{{test}}</code> · <code>predict/{{project}}</code> · <code>memory/search</code> · <code>POST /chat</code> + <code>chat/stream</code> · sessions · recommendations
reports	<code>/reports</code>	Per-run report · AI narrative · regenerate · HTML/PDF export · trends
tickets	<code>/tickets</code>	Paste issue → link PR + scenarios → auto-test on push → status flip · <code>POST /run</code>
workflow	<code>/workflow</code>	Spine + coverage matrix + per-node built/pending/manual status · <code>plan_path</code>
dependency	<code>/dependency</code>	Module graphs, blast-radius impact analysis
groups	<code>/groups</code>	Application-group CRUD + impact scope
automation	<code>/automation</code>	Device list/health, refresh, run/stop/status, <code>cancel-all-queued</code>
agents	<code>/agents</code>	<code>POST /register</code> · <code>/{{id}}/heartbeat</code>
ci	<code>/ci</code>	<code>POST /pr-autotest</code> (secret-guarded, called by GitHub Actions)
ops	<code>/ops</code>	<code>health</code> · <code>health/full</code> · <code>metrics</code> · <code>alerts</code> · <code>agents</code> · <code>audit</code>
analytics	<code>/analytics</code>	Metrics + recommendations
appium	<code>/appium</code>	Server lifecycle, session management, health, logs

Two non-REST channels

- **WebSocket** `/ws/stream/{{run_id}}` — live emulator screen. JWT validated *before* the handshake is accepted; binary frames are raw PNG at ~4 FPS; a final `{"type":"ended"}` JSON message closes it. Run IDs prefixed `test-` / `debug-` bypass JWT for the standalone stream-test page.
- **SSE** — `/scenario/run/stream` and `/intelligence/chat/stream` push per-step results and token-by-token chat.

9. The Dashboard (14 Pages)

ROUTE	PAGE	PURPOSE
/	DashboardHome	Stats, device count, scrollable Recent Test Runs with sticky header, "Run Cross-App Suite" launcher.
/projects	ProjectsPage	Project CRUD, clone/pull, prepare (with live status), validate, run.
/scenarios	ScenariosPage	The core authoring surface: build/save/edit/reorder/delete steps, pick app + device, Run and Run All, live per-step streaming, coverage tags, ✨ Suggest, engine-status chip.
/scripts	ScriptEditorPage	Monaco editor over the repo file tree; run-on-device; AI script generation grounded on captured locators.
/run/:id	RunDetails	Per-run Analysis (RCA) tab + Scenarios tab, execution timeline, evidence, live screen stream.
/reports	ReportsPage	Stored reports, AI narrative, trends over time, per-environment pass rates, flaky list, HTML/PDF export, "File Jira" button.
/pull-requests	PullRequestsPage	List open PRs per project, queue PR runs, see verdicts.
/workflow	WorkflowPage	Visual map of the whole application flow (the spine + both lanes + handoff edges) coloured by automated / pending / manual-blocked.
/graph	GraphPage	D3 module dependency graph + blast radius.
/performance	PerformancePage	CPU/memory/FPS/API latency charts, performance score, regression detection, trends.
/automation	AutomationPage	Device control: discovers local simulators directly (no agent needed), run/stop.
/chat	ChatPage	Memory-backed AI chat about the platform, runs and failures.
/settings	SettingsPage	Configuration.
/login	LoginPage	JWT login (default admin / admin123).

Shared components: `ModalPortal` (portals modals to `<body>` so they centre correctly on tall pages), `CrossAppRunModal` (role→device→credential assignment), `RecorderModal`, `MonacoEditor`, `FileTree`, `BundleIdSelect`, `ScenarioPanel`, `ChatMessage`. **API client:** `src/api.ts` — typed wrappers, `getHeaders()` for JWT, and SSE readers.

10. Core Flow A — Project Preparation

One pipeline in `projects/preparation.py` is the single source of truth for "make a repo runnable", used by the agent, the API and PR runs alike:

```
clone/pull → checkout branch → detect project type → PRE-FLIGHT VALIDATE
  → install dependencies (npm/pod/gradle) → BUILD app
  → resolve + boot target iOS device → install app → launch → READY
```

- **Pre-flight** (`utils/validator.py`) fails fast with *named fixes* — it tells you which tool is missing and how to install it rather than dying inside Xcode.
- **Detection** (`projects/detector.py`) figures out iOS / Android / React Native by looking for `ios/` , `android/` , `package.json` . Nothing is RN-specific: an RN repo simply has native projects underneath.
- **Build** (`builder.build_ios`): `pod install` → `xcodebuild` for the exact simulator → RN compatibility fixes → a `.app` . Android: `./gradlew assembleDebug` → `adb install -r` .
- The resulting artefact path is written back into `automation.yaml` as `environment.app` , so Appium can install/launch it too.
- **Metro watchdog** — started at API boot; keeps each RN packager alive on its own port so the app never shows "*No bundle URL present*".

A hard-won fix worth naming in discussion

The app used to launch *before* Metro had finished building the bundle → red "Could not connect to development server" screen. A **Metro bundle-wait** was added: the launch blocks until the packager reports ready. Similarly, each RN app needs its **own Metro port** (8081 Consumer, 8082 Business) — sharing one serves the wrong bundle.

11. Core Flow B — Plain-Language Scenario Run

This is the heart of the product: `intelligence/scenario_runner.py`, 1,389 lines, ~50 methods. A user writes:

```
open the app
open Nylai Kitchen2
select a time slot
book a table
verify the app did not crash
```

11.1 What happens per step

1. **Parse intent** — rule-based verb matching (tap / type / swipe / verify / wait / open). Deliberately *not* LLM-driven: the vocabulary a person uses to describe an app flow is small and closed.
2. **Extract the target phrase** and normalise it — `"Nylai kitchen 2"` → `"NylaiKitchen2"` (spacing + case drift).
3. **Resolve against the LIVE accessibility tree**, in escalating order:
 - **Learned locator** — the one that worked last time for this `(app, target)` pair. One fast, direct find.
 - **Exact id** match.
 - **Predicate** query on name/label/value.
 - **Container scan** — walk parent containers when the leaf isn't uniquely addressable.
 - **Fuzzy text** match with keyword/noun extraction and a "most specific element" tiebreak.
 - **LLM tie-breaker** (`pick_element`) — last resort only: "which of these on-screen ids matches this phrase" is a classification a small local model can do well.
4. **Act** — tap / type / swipe, with auto-handling of known popups (e.g. the booking popup) and LogBox dismissal.
5. **Wait for idle**, invalidate the cached page source, screenshot.
6. **Record** a real Appium/pytest line using the locator that *actually worked*.
7. **Re-learn** — persist the working locator back to `learned_locators.json`.
8. **Stream** the step result to the browser over SSE.

11.2 Assertion vocabulary

VERIFY TYPE	METHOD	WHAT IT DOES
Presence / absence	<code>_verify_step</code>	Element on screen; supports negative asserts ("did not crash").
Calculation	<code>_verify_calculation</code>	Scrapes numbers off the screen and evaluates an arithmetic expression safely.
Bill correctness	<code>_verify_bill</code> + <code>bill_validator</code>	Domain-specific: does the bill total match items + modifiers + tip?
API response	<code>_verify_api</code>	Checks the intercepted network call behind the UI action.
Size / count	<code>_verify_size</code>	List length, item counts.
Crash	<code>app_crash</code>	Detects the app dying or dropping to springboard.

11.3 Outputs of a run

- A **per-step report** (pass/fail, how the element was found, confidence).
- A generated **runnable pytest file** saved into the project's `e2e/`.
- An **element catalog** — for every step, which strategy found the element and at what confidence. This doubles as a **technical-debt report**: any step that needed the fuzzy fallback names the exact element that should get a `testID`.
- A persisted `test_run` + `scenario_results` rows + an AI summary, so the run appears in Dashboard and Reports.

12. Core Flow C — The Recorder

`api/v1/routers/recorder.py` — effectively an **in-browser Appium Inspector with record mode**. It holds one Appium session open per recording session.

1. The dashboard displays the real simulator screen as a served frame.
2. The user clicks on the mirror.
3. The backend **hit-tests** the click coordinate against the live UI tree (XML parsed with `ElementTree`) to find which element is there.
4. It **actually taps** that element through Appium, so the app advances in step with the user.
5. It **records a plain-language step** that the ScenarioRunner can replay — capturing the real `testID` where one exists.
6. Stop → save as a `SavedScenario`.

Why this matters

The recorder is what makes the platform usable by non-engineers, and it's what produces *reliable* scenarios — a recorded step carries the app's real accessibility id, whereas a hand-typed step relies on fuzzy resolution. Scenarios authored before the recorder's locator fix are flagged for re-recording.

13. Core Flow D — Cross-App Orchestration

`scenarios/cross_app_orchestrator.py` drives **two simulators at once** — Consumer on an iPhone, Business on an iPad — via two Appium sessions in parallel threads, synchronised at the cross-app handoffs:

```
order_placed → table_accepted → kitchen_accepted → payment
```

- **Roles:** Consumer / Waiter / Kitchen. Waiter + Kitchen may share one iPad (account switch) or split across two devices.
- Each phase records a `ScenarioResult` carrying both `consumer_status` and `business_status`, so the **both-must-pass gate** applies — a scenario is green only if both sides succeeded.
- Configuration (role→device→credentials) lives in `cross_app_config.py` and is edited through `CrossAppRunModal` in the dashboard. Passwords are kept local and never returned in clear.

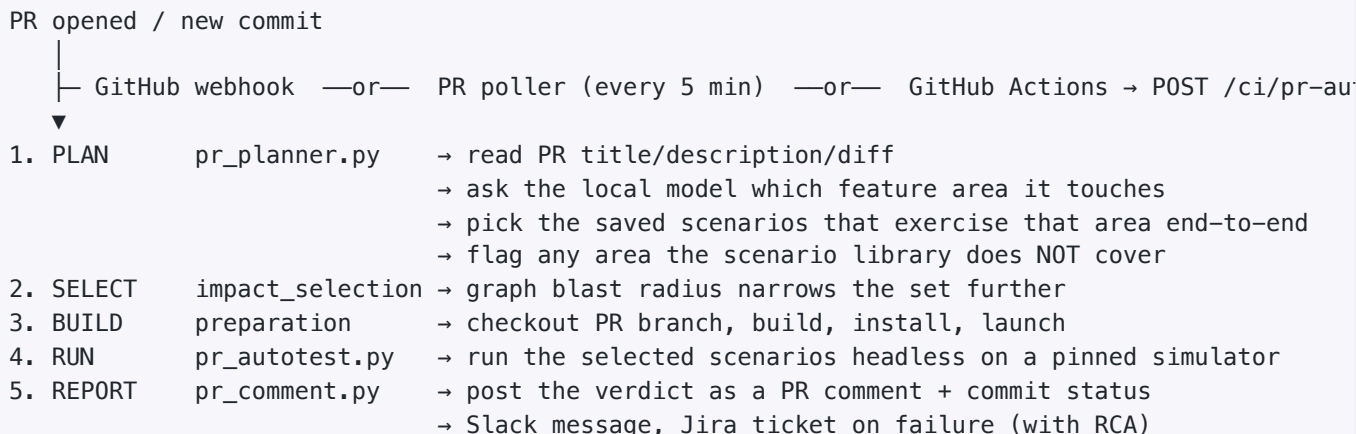
Honest limits, stated in the code itself

The orchestrator's own docstring documents why a run doesn't go fully green today: the Consumer app can crash before checkout (a TextEncoder / appointment.type issue), and the Business RN app needs its own Metro (8081 serves the Consumer bundle) so it can stall on its splash screen. The orchestrator **runs each side as far as it can and reports the truth per phase** rather than pretending. This "no false green" discipline runs through the whole platform.

The current orchestrator is **parallel**. The next build is the **sequential** flow: Consumer books → Waiter assigns table and sends to kitchen → Kitchen marks ready → Waiter serves.

14. Core Flow E — Autonomous PR Testing

The capstone that makes the platform hands-off: a developer opens a PR and gets a QA verdict where they work.



- **Device pinning:** PR runs use `PR_TEST_IOS_DEVICE` rather than "booted-wins", so results are comparable run to run.
- **Verified benefit:** ~50% reduction in scenarios executed on targeted changes, via graph-driven selection.
- **Honest verdict:** if the app has no tagged scenarios (the Business app today), the PR comment says "no tests" — it never reports a green build as verified.
- **Two ways to run it:** the **PR poller** needs nothing but `PR_AUTOTEST_ENABLED=true`; the **GitHub Actions** path additionally needs a self-hosted macOS runner (iOS simulators cannot run on GitHub-hosted runners) and gives you a proper PR status check.

Tickets — the same loop, issue-first

`tickets.py` (516 LOC) lets you paste an issue, link it to a PR and to the scenarios that verify it. When the linked PR is pushed, the platform runs those scenarios and flips the ticket's status automatically.

15. Core Flow F — The External Vya Bot

A separate repo (**Vya-agentic-BOT**, maintained alongside) runs the ~70-scenario restaurant suite on a Mac using Facebook's `idb` for coordinate-level control. The platform integrates it rather than replacing it:

1. `run_vya_bot_suite.py` (or `POST /runs/vya-bot`) creates a platform run and sets `PLATFORM_SCENARIO_CALLBACK`.
2. The bot POSTs each scenario result to `/api/v1/runs/{run_id}/scenario-result` (guarded by `X-Bot-Secret`, no JWT).
3. The platform **upserts one row per** `(run_id, scenario_num)`, merging the bot's separate Consumer and Business posts into a single row with the both-must-pass gate.
4. Results appear live in the Scenarios tab of the dashboard.

The bot-side contract is documented in `android_bot/TANISH_BOT_PATCH.md`; the platform-side reporter is `android_bot/scenario_reporter.py`. Wiring is complete; what remains is the bot's own device setup and confirmation that it reads the callback env var on every run.

Design note — why keep an external bot at all?

`idb` gives coordinate-level control that survives apps with poor accessibility ids, and the suite already existed and worked. Rather than rewrite 70 scenarios, the platform defines a **result-callback contract** and treats the bot as one more execution backend. This is the same reason there is a `plugins/` package with a `base.py` — execution frameworks are pluggable.

16. The AI Layer

16.1 Provider abstraction (`ai/provider.py` , 360 LOC)

An abstract `LLMProvider` with concrete Ollama, hosted and **mock** implementations, returning typed dataclasses (`LLMResponse` , `RCAAnalysis`). Every AI feature therefore degrades gracefully: if no model is running, you get an instant deterministic fallback rather than a 500.

16.2 What the AI actually does

FEATURE	MODULE	DESCRIPTION
RCA	<code>ai/service.py</code> → <code>RCAService</code>	Given a failed run's error, failed scenarios and evidence, produce: root cause, failure category, affected modules, confidence, possible reason, impact, suggested fix, priority, severity, responsible module, summary. Persisted to <code>rca_reports</code> and shown on the Analysis tab.
PR planner	<code>intelligence/pr_planner.py</code>	Classifies which feature area a PR touches, then picks the recorded full-path scenarios that exercise it, in run order — and explicitly flags areas the library doesn't cover.
Report narrative	<code>ai/services/reporting.py</code> , <code>summary.py</code>	Plain-English "what happened, why, what to do" for each run; PM-friendly trend summaries; cached.
Chat	<code>ai/services/chat.py</code> , <code>intelligence/chat.py</code> , <code>memory.py</code>	Memory-backed conversation over runs, failures and the platform itself, with session history and SSE streaming.
Script generation	<code>ai/services/generator.py</code>	Generates test scripts grounded on real captured locators — not hallucinated selectors.
Coverage suggest	<code>ai/services/knowledge.py</code>	The ✨ Suggest button: maps a scenario's steps to the app's real screens to auto-tag coverage.
Element tie-break	<code>scenario_runner.pick_element</code>	Last-resort classification when several on-screen elements plausibly match a phrase.

The honest caveat about AI on this machine

Ollama on a 6-core 2019 Intel CPU takes ~30–60 s per call and pins the machine. For demos the AI is kept **off** and summaries use the instant fallback. This is a hardware constraint, not an architecture one — `LLM_MODEL_NAME=llama3.1:8b` gives noticeably better report quality on adequate hardware.

17. The Intelligence Layer (The Differentiators)

17.1 Persistent self-healing locators

`intelligence/learned_locators.py` — the resolver already tried several strategies, but that knowledge died with the run. This store makes it persist:

- When a step resolves to a working locator, remember `(app, target) → locator` .
- Next run, try the remembered locator **first** — one fast direct find instead of re-deriving through fuzzy fallbacks. Faster *and* more stable.

- When the app's id drifts and the remembered locator stops resolving, fall through to the normal strategies, find it the slow way, and **re-learn**. The suite heals itself instead of going red.
- Pure and testable — no Appium, no device, thread-safe with an `RLock`. A miss just means "fall through". It has its own unit test (`tests/test_learned_locators.py`).

17.2 Hybrid cross-app impact analysis

`intelligence/hybrid_impact_analyzer.py` (516 LOC) combines two tools, each covering the other's blind spot:

TOOL	COVERS	BLIND SPOT
Graphify (tree-sitter AST)	Intra-app blast radius — which files in the same app depend on the changed file. Exact, deterministic, multi-language (Swift/TS/JS/Python/Kotlin/ObjC).	Cannot see across apps — separate repos never import each other, so it correctly reports zero edges between them.
CrossAppDependencyAnalyzer	Inter-app coupling — apps are joined only by calling the same HTTP endpoint.	Has no concept of AST reachability.

```

changed file
→ Graphify affected_nodes()      → every file in the app that depends on it
→ scan THOSE files for endpoints → the endpoints the change can touch
→ find other apps calling them   → the suites that must ALSO run

```

Using Graphify for step 2 is what makes this better than an endpoint scan alone: a change to a file that makes no HTTP call at all — a model, a constant — still reaches the services that *do* call one, because the AST walk gets there.

17.3 The rest of the intelligence layer

CAPABILITY	HOW
Flaky detection (flaky_detector.py)	Analyses the last N runs of a test, computes a confidence score, flags flakiness. Paired with flaky auto-retry and a flaky badge in the UI.
Test impact prediction (impact_predictor.py)	Risk score per test; reorders high-risk tests first so failures surface early.
Smart selection (impact_selection.py)	Turns the blast radius into a concrete scenario list. Untagged scenarios always run — selection only skips tests it can prove are unaffected.
Visual regression (visual_regression.py)	OpenCV pixel-diff of current screenshots vs stored baselines in baselines/ . Severity thresholds: >10% high, >5% medium (also the pass/fail line), >2% low.
Performance testing (performance/)	Background thread samples CPU/memory/(FPS, network) every 2 s while tests run; measures app launch + screen-load times; API calls over 500 ms flagged; produces a scored summary with regression detection. All probes best-effort — a failed probe returns None and never breaks the run.
Element / locator catalog	Per-run record of how each element was found and at what confidence — a locator-quality and technical-debt report that tells developers exactly which elements need testIDs.
Git analyzer (git_analyzer.py)	Parses the diff into changed modules feeding everything above.
Bill validator (bill_validator.py)	Domain-specific assertion engine for restaurant bill arithmetic.
Memory (memory.py)	Searchable store of past runs/failures that grounds the chat and recommendations.

18. Device & Simulator Hardening

A whole class of failures used to be "the run never started". Four layers now guarantee iOS runs begin:

1. **Registry staleness** — a device unheard-from for >90 s stops counting as ONLINE, so a dead agent's UDID is never selected.
2. **Resolve at trigger** — PR/webhook runs store a *real local* simulator UDID (preferring one already booted) instead of whatever was in the payload.
3. **Resolve + boot at prep** — `resolve_ios_device()` falls back on an invalid UDID, is platform-aware (consumer→iPhone, business→iPad when hinted), and `ensure_ios_booted()` boots a shut-down simulator automatically.
4. **Scenario runs** boot the simulator explicitly and verify Appium is reachable before connecting, with a clear error if it isn't.

Net policy: booted wins; if nothing is booted, boot one automatically.

Appium self-healing & orphan reaping

- If WebDriverAgent wedges ("device not working"), the next run clears and restarts it automatically. The Scenarios tab shows an **engine-status chip** (ready / will auto-start).
- On API startup, `_reap_orphaned_runs()` marks any run still `running` after a restart as failed/cancelled — this is what removed the pile of ghost "RUNNING" rows.
- First run after a restart rebuilds WebDriverAgent (~20–30 s, up to 3–5 min cold on the Intel Mac). This is expected; the live log says so rather than looking like a hang.

19. Reporting, Notifications & Integrations

CAPABILITY	DETAIL
Stored reports	Every run aggregates metadata + per-scenario results + RCA into one stored report. Regenerable; exportable as HTML and PDF .
AI narrative	Plain-English "what happened, why, what to do" per run.
Trends	Pass rate over time, per-environment breakdown, flaky list, performance trends.
Honest verdicts	A run with zero scenarios reports "no tests" — never a false green. This is enforced at report level, PR-comment level and orchestrator level.
Slack	Message on every PR-QA verdict + real-time failure alerts. Needs <code>SLACK_WEBHOOK_URL</code> .
Jira	Auto-ticket on failure carrying the RCA, plus a manual "File Jira" button on any report. Needs <code>JIRA_URL/EMAIL/TOKEN/PROJECT_KEY</code> .
GitHub	PR listing, PR comments with the verdict, commit status, webhooks for push/PR, clone URL resolution. Needs <code>GITHUB_TOKEN</code> , <code>GITHUB_REPO_OWNER</code> .
Evidence	Screenshots + logs bundled per run, PII-scrubbed before storage.
Live streaming	The emulator screen streamed to the browser at ~4 FPS over WebSocket while a run executes.
Secret masking	<code>install_secret_filter()</code> at startup redacts tokens and keys from <i>all</i> logs.

20. Operations — Running the Platform

20.1 One command

```
./start.sh      # boots simulators → Appium → SUPERVISED backend → dashboard
./status.sh     # what's up
./stop.sh       # tear it all down
```

`start.sh` is idempotent and does the following in order:

1. Reads simulator UDIDs from `cross_app_config.json`, opens Simulator.app, boots each.
2. Starts **Appium** on 4723 with `--relaxed-security` and a 4 GB Node heap (Metro OOM'd without it), waits for `"ready":true`.
3. Starts the backend under `scripts/supervise_backend.sh` — a supervisor that **auto-restarts it on crash** — and waits for `/docs` to return 200.
4. Starts the Vite dashboard on 5173.
5. Ensures `node` from `nvm` is on PATH, because Appium and Metro shebangs are `#!/usr/bin/env node` and a detached process without it dies with "env: node: not found".

Never run the backend with `uvicorn --reload`

`--reload` watches the whole project, and the platform *writes files during runs* (DB, screenshots, builds, graph cache). It would restart mid-run and kill everything. Use `./start.sh` (or `./start-backend.sh`); a scoped reload form for development is documented in the script's comments.

20.2 Environment configuration (`.env`)

Every feature no-ops safely until configured — nothing hard-fails on a missing key.

```
PR_AUTOTEST_ENABLED=true           # run the honest smart-selected autotest on new PR commits
PR_TEST_IOS_DEVICE=<UDID>          # pin PR runs to one simulator
SLACK_WEBHOOK_URL=...              # verdicts + real-time failure alerts
JIRA_URL / JIRA_EMAIL / JIRA_TOKEN / JIRA_PROJECT_KEY
CI_SECRET=...                      # must match the GitHub repo secret
GITHUB_TOKEN / GITHUB_REPO_OWNER
AGENT_SECRET / BOT_SECRET          # guard the frame-upload and result-callback endpoints
DASHBOARD_URL=http://localhost:5173
LLM_MODEL_NAME=llama3.1:8b         # better narrative quality than the default llama3.2
DATABASE_URL=...                   # switch SQLite → Postgres
```

A sanitised `.env.example` is committed; the real `.env` and the `.db` files were untracked in the most recent commit (`da49069 security: untrack .env + .db files`).

20.3 Logs

`logs/` holds `backend.log`, `supervisor.log`, `dashboard.log`, `appium.log`, and per-session Appium logs under `logs/appium/` (including pre-flight sessions).

21. Current Status — Honest Scorecard

AREA	STATUS	NOTE / WHAT'S NEEDED
iOS execution (Appium / XCUITest / WDA)	DONE	Works; first run pays the WDA build cost.
Scenario recorder	DONE	Records real testIDs.
Scenario run / Run All / self-healing	DONE	Persistent learned locators.
Coverage tags + AI Suggest	DONE	Needs someone to click Suggest per scenario.
Git change analyzer	DONE	—
Graph-driven smart test selection	DONE	~50% reduction verified on targeted changes.
PR planner + autonomous PR loop	DONE	Enable with <code>PR_AUTOTEST_ENABLED=true</code> .
RCA + reports + trends + PDF/HTML	DONE	—
Honest verdicts (no false green)	DONE	—
Visual regression	DONE	Baselines need capturing per screen.
Test impact prediction	DONE	—
Performance testing (CPU/mem/FPS/API + score)	DONE	657 metrics already collected.
Flaky detection + auto-retry	DONE	—
Live screen streaming	DONE	Frame polling at 4 FPS; true WS push would be smoother.
Vya bot integration (~70 scenarios)	PARTIAL	Wiring complete; needs the bot's device setup + confirmation it reads the callback env var each run.
Slack notifications	PARTIAL	Set <code>SLACK_WEBHOOK_URL</code> .
Jira auto-tickets	PARTIAL	Set the <code>JIRA_*</code> values.
CI (GitHub Actions)	PARTIAL	Needs a self-hosted macOS runner + repo secrets. The PR poller already works without it.
Environments (prod / staging)	PARTIAL	Staging projects registered; needs per-app Metro port pinning + staging credentials.
Scenario library per app	PARTIAL	The biggest lever. Consumer has scenarios; the Business app had 0 — which is why its PRs honestly report "no tests".
Sequential cross-app flow	NOT DONE	Current orchestrator is parallel. Next build.
Postgres / Docker / hosting	NOT DONE	<code>docker-compose.yml</code> exists but isn't wired. Dashboard API base is hardcoded to localhost.
Jenkins / Azure pipelines	NOT DONE	YAML stubs only.
Platform's own test coverage	THIN	~4 tests today (<code>automation/tests/</code>).

22. Real Numbers From the Live Database

183 TEST RUNS	161 SCENARIO RESULTS	15 SAVED SCENARIOS	9 PROJECTS	25 RCA REPORTS
657 PERF METRICS	32 EVIDENCE BUNDLES	169 SYSTEM ALERTS	40 AGENT REGISTRATIONS	10 DEPENDENCY GRAPHS

Registered projects (9)

- Consumer App — `vya-consumer`
- Business App — `vya-business`
- Vya web App — `vya_web_app`
- Vya Staging Consumer / Business / Web
- Food Delivery (third-party sample)
- iOS Settings Tests x2 (local demo repo)

Saved scenarios (15)

- **Consumer:** book a table · Smoke — app launches · Browse & add to cart · Preorder a time slot · Reach checkout & pay · Navigate to wallet · Book an event
- **Business:** Login as waiter · Book event (waiter) · Assign table + add item + send to kitchen · Kitchen mark ready · Serve + pay + close
- **XP/gamification:** Pre-book +20 XP · Cancel -30 XP · Verify XP shows correctly

Run outcomes to date

STATUS	COUNT	READING
failed	134	Includes every iteration during engine development — device setup, Metro, WDA, locator drift. Not a product pass-rate.
passed	17	—
cancelled	17	—
stopped	9	—
queued / pending	6	—

How to present this number honestly: the database is a development log, not a production QA record. The failure count is the history of building the execution engine on a constrained machine; the meaningful signal is that individual scenarios now run and pass live step-by-step (e.g. "book a table" went from 5/6 steps to clean).

Codebase size

METRIC	VALUE
Python files / LOC	134 files · ~24,700 lines
TypeScript + TSX LOC (dashboard)	~9,800 lines
API router files / route decorators	22 routers · 110+ endpoints
Database tables	24
Largest modules	<code>builder.py</code> 1,473 · <code>scenario_runner.py</code> 1,389 · <code>jobs.py</code> 821 · <code>projects.py</code> 725 · <code>scenario.py</code> 667

23. Known Issues & Root Causes

These are the war stories — the most valuable thing to be able to explain in a discussion, because each one is a real debugging narrative with a root cause and a fix.

SYMPTOM	ROOT CAUSE	RESOLUTION
"No element matches ..." on every step	Appium's UI snapshot was capped at depth 40 , which hid deeply-nested React Native elements — Appium literally could not see them.	Raised the snapshot depth to 60 . FIXED
Red screen: "Could not connect to development server"	The app launched before Metro finished building the bundle.	Added a Metro bundle-wait before launch. FIXED
"Connecting... building WebDriverAgent" for minutes	Cold WDA build on a 2019 Intel Mac.	Pre-built and cached; warm up once before a demo. HARDWARE
Everything crawls during a run	Ollama pinning all 6 CPU cores.	Stop Ollama for live runs; AI falls back instantly. FIXED
Step <code>select preorder</code> always failed	The app UI is in German ("Vorbestellung") and the step duplicated an automatic popup handler.	Removed the duplicate step. FIXED
Locator misses on names like "Nylai kitchen 2"	Spacing and case drift vs the real testID <code>NylaiKitchen2</code> .	Locator normalisation in the resolver. FIXED
"No connected devices" on the Automation page	Only agent-fed devices were listed; no agent was running.	Discover local simulators directly. FIXED
Scenario runs missing from Reports	Scenarios-tab runs weren't persisted as <code>test_run</code> rows.	Now saved with steps + AI summary. FIXED
Pile of ghost "RUNNING" rows	A backend restart killed run processes but left rows at <code>running</code> .	Orphan reaping at startup. FIXED
Business app stalls on splash / shows the wrong bundle	Port 8081 serves the <i>Consumer</i> bundle; the Business app needs its own Metro.	Per-app Metro ports (8082) + <code>RCT_jsLocation</code> ; a reload resolves it. FIXED
Modals mis-centred on tall pages	Modals rendered inside a scrolled container.	<code>ModalPortal</code> renders them into <code><body></code> . FIXED
Cross-app run doesn't go fully green	Consumer app can crash before checkout (TextEncoder / appointment.type); Business login is account-gated.	App-side blockers; the orchestrator reports the truth per phase. OPEN
Business app PRs show "no tests"	Zero recorded scenarios for that app.	Record its key flows. This is the top manual action. OPEN
Leaked GitHub token in git history	Token committed before <code>.env</code> was untracked.	<code>.env</code> / <code>.db</code> now untracked + <code>.env.example</code> added; the token still needs rotating on GitHub . ACTION REQUIRED

24. The Roadmap — Layered Automation

The governing principle: don't automate 150 scenarios at once. Build a **dependency chain** where each scenario reuses the building blocks below it, verify each block live before building the next, and later categories become small *variations* of the core chain.

LAYER	CONTENTS	STATUS
0 — Foundations	Consumer: book an event · Business: login as waiter · login as kitchen · Consumer login	Booking built and ~working; the logins are blocked on valid credentials .
1 — The spine	Waiter books → assign table → add item → send to kitchen → kitchen marks ready → serve → notify payment → pay cash → close table	Pending Layer 0 credentials. When Layer 1 runs end-to-end, the full flow exists.
2 — Booking variations	1 guest / 1 participant / both · indoor / outdoor · 2 h / 3 h · accept / decline · modify event	Variations of Layer 0–1.
3 — Ordering variations	Modifiers · variants · special instructions · edit items · reassign item · split before/after serve · tips	Reuses 1.2–1.4.
4 — Preorder	Cart add/reduce/edit during event · host vs invitee preorders · coupon add/remove + total verify	Reuses 0.1.
5 — Payments	Cash / e-payment / food voucher (+ decimal tip) · PDF check · host pays all · participant pays for others · split / mixed · remind payment + re-checkout · combined B-App + C-App	Reuses the full spine.
6 — Cancellations & status	Cancel from either app · before/after 15 min · me-only vs cancel-all · after table assigned · pickup cancellation · status verification (No-Show, Preorder, Cancelled, Completed, Payment Requested)	—
7 — Long tail	Contacts · subscriptions · block/unblock · filters · PDF generation · complimentary items · void items (before/after send, over-void error)	—

Production hardening (when moving off this Mac)

1. Rotate the leaked GitHub token — highest priority.
2. Postgres + Alembic replacing SQLite.
3. Make the dashboard's API base an **env var** (currently hardcoded to `localhost:8000`).
4. Containerise + process supervision (launchd/docker) + a real `/health` contract.
5. CI on the platform itself + real test coverage (~4 tests today).
6. Configuration instead of hardcoded paths/UDIDs.

The hosting reality — say this clearly if asked

Everything runs **locally on a Mac** today, because iOS simulators require macOS + Xcode. The Render URL serves **only the frontend**; the API is not hosted there. Even after hosting the API and moving to Postgres, **the bot, Appium and the simulators must stay on a Mac**. That is a property of iOS, not a shortcut in this design.

25. Design Decisions & Trade-offs

DECISION	WHY	TRADE-OFF ACCEPTED
Rule-based step parsing, LLM only as tie-breaker	The vocabulary people use to describe an app flow is small and closed. Rules are deterministic, instant and debuggable.	Unusual phrasings need a new rule rather than "the model figures it out".
Local LLM first (Ollama), mock fallback	Zero API keys, zero data leaving the machine — test logs contain app internals and credentials.	Slow on CPU; quality below hosted frontier models.
Resolve against the live accessibility tree, not stored selectors	Selectors written by hand rot; the live tree is ground truth.	Every step costs a page-source fetch (mitigated by a 1.5 s TTL cache).
Persist learned locators to disk	Turns per-run self-healing into cross-run memory: faster <i>and</i> more stable.	A stale entry costs one failed find before falling through — bounded and safe.
Both-must-pass gating on cross-app scenarios	A cross-app scenario is only meaningful if both sides succeed.	Green is harder to achieve — deliberately.
Honest verdicts over green dashboards	A false green is worse than a red: it destroys trust in the whole system.	The dashboard looks worse than a naive one would.
SQLite with auto column-add migration	Zero-setup development; schema changes never block a run.	Single-writer; must move to Postgres for multi-user/hosted use.
Keep the external <code>idb</code> bot instead of rewriting it	70 working scenarios already existed; <code>idb</code> handles apps with poor accessibility ids.	Two execution backends to maintain; integration via a callback contract.
Supervised backend, no <code>--reload</code>	The platform writes files during runs; a reloader would kill runs mid-flight.	Code changes need a restart (the supervisor makes that one command).
Graphify AST + endpoint scan hybrid	Neither alone can express "a change in app A breaks a flow in app B".	Two systems to keep in sync; the endpoint layer is heuristic.

26. Group Discussion Q&A Prep

Q. In one sentence, what is this?

An AI-assisted mobile QA platform that turns plain-English steps into real Appium runs on iOS simulators across two interacting apps, and posts an automatic QA verdict back on every pull request.

Q. How is this different from just writing Appium tests?

Three ways. (1) **Authoring** — steps are plain English resolved against the live accessibility tree, so a non-engineer can author and the platform generates the real script. (2) **Durability** — learned locators persist and re-learn on drift, so the suite heals instead of going red. (3) **Scope** — it orchestrates two apps on two simulators with both-must-pass gating, which a plain Appium script cannot express.

Q. What's the hardest technical problem you solved?

Element resolution in React Native. Appium's default UI snapshot depth of 40 silently hid deeply nested RN elements, so every step failed with "no element matches" — the symptom looked like a locator bug but was a visibility limit. Raising it to 60, plus locator normalisation ("Nylai kitchen 2" → "NylaiKitchen2") and a Metro bundle-wait before launch, is what turned the engine from failing to working.

Q. Why not use a frontier LLM to drive the whole thing?

Because determinism matters more than cleverness here, and test logs contain credentials and app internals. The design is local-first with a provider abstraction: rules do the driving, the model only classifies ("which of these on-screen ids matches this phrase") and writes narratives. Swapping in a hosted model is a config change, not a rewrite.

Q. How do you keep a big suite fast?

Graph-driven smart selection. From the git diff, Graphify's AST walk gives the intra-app blast radius; those files are scanned for HTTP endpoints; other apps calling the same endpoints are pulled in. That yields the scenario set to run — verified around a 50% reduction on targeted changes. Crucially, **untagged scenarios always run**: we only skip what we can prove is unaffected.

Q. What's the weakest part right now?

The scenario library, not the engine. The Business app had zero recorded scenarios, which is why its PRs honestly report "no tests". Everything downstream — smart selection, autotest, reports, the reduction percentage — is only as good as the tagged library. That's the main ongoing manual work.

Q. Your database shows 134 failed runs. Isn't that bad?

That's a development log, not a QA pass-rate. Those 183 runs are the history of building the execution engine on a constrained Intel Mac — device setup, Metro, WebDriverAgent, locator drift. The meaningful signal is that individual scenarios now run and pass live, step by step.

Q. Why does the platform sometimes take 3–5 minutes to start a run?

A cold WebDriverAgent build on a 2019 Intel Mac. It's a one-time cost per session and it's Xcode, not the platform — WDA is pre-built and cached, and the live log tells the user it's expected rather than looking like a hang. On modern hardware it's ~20–30 seconds.

Q. Can this be deployed to the cloud?

The API and dashboard can — that needs a public FastAPI deploy, the dashboard's API base moved to an env var, and SQLite → Postgres. But Appium, the simulators and the bot **must stay on a Mac**, because iOS simulators require macOS and Xcode. The realistic architecture is a hosted control plane plus Mac execution nodes — which is exactly what the agent/job-polling design already anticipates.

Q. What does "honest verdict" mean and why do you keep saying it?

A run with zero scenarios reports "no tests", not "passed". A cross-app run reports the truth per phase rather than hiding a blocked side. A false green is worse than a red because it destroys trust in every other result the system produces — so the principle is enforced at report, PR-comment and orchestrator level, and un-automatable flows are explicitly tagged MANUAL.

Q. How does the self-healing actually work?

On a successful step we store (app, target) → locator on disk. Next run we try that locator first — one direct find instead of the fuzzy cascade. If the developer renames the element, that find fails, we fall through to exact-id → predicate → container scan → fuzzy text → LLM tie-break, find it the slow way, and overwrite the memory with the new locator. A miss only ever costs one wasted find, so it's safe by construction.

Q. What would you build next, given a week?

The sequential cross-app flow (Consumer books → Waiter assigns and sends to kitchen → Kitchen marks ready → Waiter serves and closes), because it unlocks Layers 2–7 as cheap variations; and recording the Business app's scenario library, because it's the single highest-leverage manual task. Then rotate the leaked token and move to Postgres.

Q. What did you personally learn?

That in test automation the hard problems are almost never the framework — they're timing (Metro vs launch), visibility (snapshot depth), identity (locator drift), and trust (never showing a false green). Every major fix in this project was one of those four, not an Appium API question.

27. Glossary

TERM	MEANING
Appium	Open-source WebDriver server that drives native mobile apps. Runs on :4723.
WebDriverAgent (WDA)	Apple's XCUITest agent that Appium installs on the simulator to actually perform taps. Its cold build is the slow first-run cost.
XCUITest	Apple's native UI testing framework; the driver Appium uses on iOS.
Metro	React Native's JavaScript bundler. Each RN app needs its own port or it serves the wrong bundle.
simctl	<code>xcrun simctl</code> — Apple's CLI for booting, installing to and screenshotting simulators.
idb	Facebook's iOS Development Bridge — coordinate-level simulator control; used by the external bot.
adb	Android Debug Bridge — the Android equivalent for device discovery and APK install.
testID / accessibility id	The stable identifier a developer puts on an element. Its presence is the difference between a reliable and a fuzzy-matched step.
Accessibility tree	The live hierarchy of on-screen elements Appium exposes as XML — the ground truth the resolver matches against.
Locator	The strategy + value used to find an element (id, predicate, xpath, text).
Self-healing	Re-deriving a locator automatically when the remembered one stops resolving.
Flaky test	A test that passes and fails on the same code. Detected from run history and given a confidence score.
RCA	Root Cause Analysis — the AI-generated explanation of why a run failed.
Blast radius	The set of files/modules/apps a code change can affect.
Graphify	Tree-sitter based tool that builds a multi-language AST dependency graph; cached in <code>graphify-out/</code> .
Smart selection	Running only the scenarios the blast radius implies, instead of the whole suite.
Both-must-pass gate	A cross-app scenario is green only if both the Consumer and Business sides passed.
Honest verdict	Reporting "no tests" rather than "passed" when nothing was actually verified.
The spine	The core end-to-end cross-app happy path every other scenario is a variation of.
Evidence bundle	Screenshots + logs captured for a run, PII-scrubbed before storage.
Execution agent	A worker process that registers with the API, heartbeats, polls for jobs and runs them against its local devices.
SSE	Server-Sent Events — how per-step results and chat tokens stream to the browser.
POM	Page Object Model — the classic test-structuring pattern used in the sample repo <code>demo-mobile-tests/</code> .

If you remember only five things

1. **Plain English in, real Appium script out** — resolved against the live accessibility tree, never a guessed selector.
2. **Cross-app is the point** — two simulators, two roles, synchronised handoffs, both-must-pass gating.
3. **Self-healing locators persist** — the suite gets faster and more stable with every run instead of rotting.
4. **Smart selection from the git diff** — AST blast radius + endpoint coupling, ~50% fewer scenarios on targeted changes.
5. **Honest verdicts** — the system would rather show red or "no tests" than a green it can't defend.